CASE STUDY · 2026

PrimarIA

AI Platform for Romanian Municipalities

A dual-portal, role-scoped AI system that turns paper-heavy citizen and admin workflows into auditable, structured operations — built for the constraints of public-sector deployment.

Author: Vasile Bogdan Godja

Role: Founder, sole engineer

Stack: Python · FastAPI · Next.js · PostgreSQL · pgvector · Redis · Docker

LLM layer: Claude · OpenAI · Gemini · local models, role-scoped routing

Status: In active development · Romanian municipal pilot scope

Contact: godjabogdan@gmail.com · github.com/Bogdan0708

---

01 The Problem

Romanian municipalities run on paper. Citizens face opaque processes; employees burn their days on classification and form-filling; mayors have no live visibility on what's moving through their administration. None of this is a technology gap — it's a deployment gap.

What the citizen faces

- **Opaque processes.** Whether a permit, tax certificate, or social benefit, the citizen has no idea what stage their request is in or what's blocking it.
- **Counter-only access.** Most interactions still require a physical visit to the mayoralty during business hours — a hard problem for working citizens and rural communes.
- **Inconsistent answers.** The answer to the same question depends on which employee picks up the desk that day.
- **No status visibility.** "Come back next week" is the default response. There's no portal, no SMS, no email update path.

What the employee faces

- **Repetitive intake.** 60–80% of incoming requests are minor variations on the same 20 categories. Each is hand-classified, hand-routed, hand-answered.
- **Manual document processing.** Scanned IDs, certificates, and applications come in as PDFs or photos. Extraction is manual; mistakes mean the file ping-pongs for weeks.
- **Audit anxiety.** Public-sector decisions need a paper trail. Most current systems don't capture *why* a decision was made, so audits become forensic exercises.
- **No tooling.** The few digital tools that exist are 15+ years old, single-purpose, and don't talk to each other.

Why existing approaches fail

National e-government portals

Cover central-government services well but stop at the municipality boundary. The actual citizen-municipality interaction remains analog.

Private GovTech vendors

Sell document-management systems that digitise the filing cabinet without changing the workflow. The employee still does the same work, now with extra clicks.

Generic LLM chatbots

Hallucinate eligibility, can't cite source regulations, and have no audit trail. Disqualifying for public-sector use the first time a citizen acts on bad advice.

Internal scripts

A handful of mayoralties have technically-minded employees writing Excel macros. Brittle, undocumented, and lost the moment the employee leaves.

Why this is the right moment

Three things have converged: LLMs that can reliably handle Romanian (a long-time gap), structured-output tooling mature enough for compliance-grade workflows, and a generational handover in Romanian local government where younger administrators actively want digital tooling and have small but real budgets to fund it.

> *PrimarIA is built on the assumption that the bottleneck is not AI capability — it's role-scoped deployment, audit trails, and integration with the messy realities of Romanian public-sector work.*

02 The Solution

PrimarIA is a dual-portal AI platform: a citizen-facing taxpayer portal and an employee/admin workspace, both driven by role-scoped agents with shared infrastructure and complete audit trails.

Two surfaces, one platform

```
┌────────────────────────────────────┐    ┌────────────────────────────────────┐
│    CITIZEN PORTAL                  │    │    ADMIN WORKSPACE                 │
│                                    │    │                                    │
│  • Tax & fee status                │    │  • Inbox triage                    │
│  • Document submission             │    │  • Document classification         │
│  • Permit / certificate requests   │    │  • Response drafting               │
│  • Status tracking                 │    │  • Audit log & decision trail      │
│  • Multilingual help (RO + EN)     │    │  • Bulk operations                 │
│                                    │    │                                    │
│  Public-grade UX, conservative     │    │  Power-tool UX, expert defaults    │
└────────────────────────────────────┘    └────────────────────────────────────┘
                 │                                          │
                 └───────────────┬──────────────────────────┘
                                 ▼
              ┌────────────────────────────────────────┐
              │        Role-scoped agent layer         │
              │        + shared retrieval + audit      │
              └────────────────────────────────────────┘
```

The four core agents

Intake agent
Classifies every incoming citizen request into one of ~40 service categories, extracts structured fields (ID, address, request type, attached docs), and routes to the right department queue.

Document agent
OCR + structured extraction over the messy reality of Romanian municipal paperwork: scanned IDs, hand-stamped certificates, mixed-quality phone photos. Outputs a typed record, not free text.

Drafting agent
Produces first-draft official responses citing the relevant regulation. Employees review and approve; the system never sends on its own.

Citizen-help agent
Conservative-by-default citizen chat. Answers procedural questions, explains required documents, links to live status. Refuses to answer anything it can't ground in source.

Role-scoping as a first-class concept

Every agent runs under a typed role context. A citizen-portal agent literally cannot invoke admin-only tools — not because the prompt says so, but because the orchestrator enforces the boundary at the infrastructure layer. This matters because in a public-sector deployment, "the model decided not to" is not an acceptable security answer.

Key product decisions

- **Conservative-by-default citizen surface.** Citizen-facing answers are restricted to what the system can cite. Anything ambiguous is escalated to a human, never guessed.
- **Structured outputs for every decision.** Classifications, extractions, eligibility flags, and routing decisions are all JSON-schema-validated.
- **Audit trail at the agent level, not the API level.** Every model call is logged with role, prompt, retrieved evidence, output, validator status, and human override (if any).
- **Romanian-first language posture.** All UX, prompts, evaluation sets, and retrieval corpora are Romanian-native. English is a translation layer, not the primary surface.

- **Multi-tenancy from day one.** Designed to deploy per-municipality with isolated data, shared infrastructure, and per-tenant configuration of categories, departments, and response templates.

What we deliberately did *not* build

- No autonomous decision-making. Every employee-facing draft is reviewed; every citizen-facing answer is grounded or escalated.
- No "AI mayor" replacing human judgement. PrimarIA accelerates routine work and surfaces decisions to the right human, never substitutes for them.
- No bespoke fine-tuning per municipality. Configuration handles 95% of local variance; the remaining 5% is template, not model.

03 Architecture

PrimarIA is a multi-tenant FastAPI + Next.js platform with three load-bearing layers: role-scoped agent execution, retrieval over per-municipality corpora, and a tamper-evident audit log. Public-sector defaults throughout.

High-level system diagram

```
┌──────────────────────────┐                      ┌──────────────────────────┐
│      Citizen Portal      │                      │      Admin Workspace     │
│   (Next.js · public)     │                      │   (Next.js · auth'd)     │
└────────────┬─────────────┘                      └────────────┬─────────────┘
             │ REST + SSE                                      │ REST + SSE
             ▼                                                 ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          FastAPI Backend                                 │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐    │
│  │ Role-scoped      │  │ Multi-LLM        │  │ Audit Log            │    │
│  │ agent runtime    │  │ orchestrator     │  │ (append-only)        │    │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘    │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐    │
│  │ Retrieval layer  │  │ Document agent   │  │ Multi-tenancy +      │    │
│  │ (pgvector+BM25)  │  │ (OCR + extract)  │  │ per-tenant config    │    │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘    │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────────┐    │
│  │ PostgreSQL       │  │ Redis            │  │ Object storage       │    │
│  │ (state, audit)   │  │ (jobs, cache)    │  │ (documents)          │    │
│  └──────────────────┘  └──────────────────┘  └──────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
                          Docker · containerised
                          deployment (cloud or on-prem)
```

Stack

Python 3.12 · FastAPI · Next.js (App Router) · TypeScript · PostgreSQL · pgvector · Redis · Docker · Claude (Anthropic) · OpenAI · Gemini · Local LLMs (Ollama) · OCR (Tesseract / cloud) · Pydantic · SQLAlchemy

Role-scoped agent runtime

```
  incoming request
        │
        ▼
┌──────────────────┐
│ Role context     │   role = citizen | employee | admin | auditor
│ (typed)          │   tenant = municipality_id
└────────┬─────────┘
         │
         ▼
┌──────────────────┐    permitted tools only
│ Agent            │  ──────────────────────────►  ┌──────────────────┐
│ dispatcher       │                               │  Tool registry   │
└────────┬─────────┘  ◄── refuses unscoped ─────── │ (per-role allow) │
         │                                         └──────────────────┘
         ▼
┌──────────────────┐
│ LLM call         │   provider chosen per task
│ (structured)     │   schema enforced
└────────┬─────────┘
         │
         ▼
```

```
┌────────────────┐
│  Audit writer  │   role · tenant · tool · evidence · output · approver
└────────────────┘
```

Multi-provider LLM routing

Task	Default provider	Why	Fallback
Intake classification	Local LLM (Ollama)	Cheap, private, fast on commodity hardware	Gemini Flash
Document extraction	Claude Sonnet	Best on structured outputs over OCR text	OpenAI structured-out
Drafting (employee-facing)	Claude Opus	Quality of Romanian legal/administrative prose	OpenAI / Gemini Pro
Citizen-help chat	Gemini Flash	Latency + cost at public scale	OpenAI mini
Embeddings	Local BGE	Private by default for sensitive municipal data	OpenAI for non-sensitive corpora

Retrieval over municipal corpora

1. **Ingestion.** Local council resolutions, national regulations, municipal templates, and historical responses are chunked and indexed per tenant.
2. **Indexing.** Hybrid pgvector + BM25 with document-type and effective-date metadata. Regulation lookups must respect "currently in force."
3. **Retrieval.** Tenant-scoped queries with mandatory date filtering. A regulation rescinded last year cannot influence today's answer.
4. **Citation enforcement.** Both employee-facing drafts and citizen-facing chat are required to cite the regulation or template they reference.
5. **Fallback to human.** If retrieval returns no high-confidence match, the system refuses to answer and routes to a named employee.

04 Technical Challenges

The interesting work in PrimarIA was not the LLM. It was the boundary between AI reasoning and public-sector reality: role-scoping, audit trails, OCR over messy real-world documents, and Romanian-language reliability.

1. Role-scoping that survives a security review

The first iteration enforced role boundaries through prompts ("you are a citizen-facing agent, never reveal admin data"). That works for demos and fails for adversaries. The fix was to make role-scoping an infrastructure property: the tool registry rejects calls outside the role's allowlist, the retrieval layer scopes queries to the citizen-visible corpus, and the audit log captures every refusal. No prompt bypass exists because there is nothing in the prompt to bypass.

2. OCR over the reality of Romanian municipal paperwork

Real submissions are phone photos of stamped certificates, scanned IDs at 150dpi, and hand-completed forms with crossed-out fields. Naive OCR + structured extraction succeeded on demo data and broke on the third real document. The fix was a two-stage pipeline: OCR with multiple engines (compare confidence), then an extraction agent that explicitly returns "unreadable" with a region pointer when confidence is below threshold — kicking the document to a human reviewer rather than silently fabricating a field.

3. Romanian-language reliability

Modern LLMs handle Romanian well but not identically across providers. The discipline that worked: per-task language evaluation sets in Romanian, not just English translated. A classification accuracy of 92% on English documents was 78% on the equivalent Romanian set. Routing decisions in PrimarIA reflect those measured gaps, not vendor marketing.

4. Tamper-evident audit log

Public-sector auditors need to verify that an AI-assisted decision was not retroactively rewritten. The audit log is append-only with hash-chained entries: each entry includes the hash of the previous entry, so any tampering invalidates the chain from that point forward. The chain is exportable for external verification. None of this is cryptographic novelty — it's standard discipline applied where it matters.

5. Multi-tenancy without explosion of operational surface

Per-municipality isolation is non-negotiable, but per-municipality infrastructure would sink the unit economics. The compromise: shared application layer, per-tenant database schema, per-tenant retrieval corpora, per-tenant configuration tables. New municipality onboarding is a config + corpus ingest, not a deployment.

6. Hallucination mitigation for compliance-grade content

Same discipline as EuFund: structured outputs with citation requirements, validator retries on schema failure, and explicit "unable to answer" pathways. Public-sector content is the worst possible failure domain for free-text hallucination — getting eligibility wrong on a permit application is materially different from getting a movie recommendation wrong.

7. Deployment topology

The architecture supports both cloud and on-premise deployment because some municipalities have data-residency requirements that rule out hosted infrastructure. Local LLMs via Ollama matter here: they make on-premise deployment cost-viable for classification and extraction workloads. Cloud LLMs handle the long-tail of difficult drafting tasks where quality dominates.

Failure mode	Mitigation
Role boundary bypass via prompt	Tool allowlists enforced in dispatcher; retrieval scoped at infra layer
Fabricated extraction over poor OCR	Confidence thresholds + explicit "unreadable" path with region pointer
Romanian-language quality drift	Per-task RO evaluation sets; routing decisions calibrated against measured accuracy

Failure mode	Mitigation
Tampered audit log	Hash-chained append-only entries; exportable for external verification
Stale regulation in retrieval	Effective-date metadata; mandatory date filter on regulation queries
Multi-tenant data leak	Per-tenant schema isolation; tenant_id in every query at ORM layer
On-premise cost	Local LLMs for classification/extraction; cloud only for high-value drafting

05 Business Impact & Lessons

Where PrimarIA creates value

2	4	100%	RO-first
Distinct portals on shared infra	Core agents (intake, doc, draft, help)	Of decisions captured in audit chain	Language posture across UX + evals

Who PrimarIA is for

- **Romanian municipalities** (10k–250k population) with small IT capacity and growing service expectations.
- **Mayoralty digital-transformation leads** who need a deployable platform, not a 12-month custom build.
- **National- and EU-level civic-tech programmes** looking for a multi-tenant template they can fund and scale.

Scale-out story

Romania first because the regulatory environment is familiar and the workflow gap is visible. The architecture is designed for multi-tenancy and country-specific configuration, so expansion to neighbouring CEE countries is a corpus + template problem, not a re-architecture.

Lessons learned

1. Role-scoping must be infrastructure, not prompting.

Every public-sector AI system is one social-engineering prompt away from disaster if role boundaries live in instructions. Move them into the tool registry and retrieval layer; treat the prompt as untrusted user input.

2. "Refuse to answer" is a product feature.

The most-praised behaviour in early testing wasn't a clever answer — it was the system politely refusing when it didn't have grounded evidence and routing the citizen to a named human. Conservatism builds trust in this domain.

3. Audit trail is the deployment unlock.

Without a hash-chained audit log, no municipal lawyer signs off. With one, the conversation moves from "can we deploy AI" to "which workflow first." The audit layer is small to build and disproportionately valuable in the sales conversation.

4. Local LLMs are economic infrastructure, not science projects.

For high-volume, low-stakes tasks (classification, extraction), local models on commodity hardware are not just cheaper — they're a deployment unlock for any municipality with data-residency constraints. The architecture treats local and cloud as equal-class providers.

5. Romanian-first matters more than English-translated.

Building UX and evals in Romanian from day one surfaced model behaviour gaps that English-first development would have hidden until production. The cost is one extra evaluation set per task; the benefit is calibrated routing.

What I'd do differently from day one

- Treat the audit log as infrastructure from commit #1, not after the third demo.
- Build the per-tenant configuration model before the second municipality, not after onboarding pain forces it.
- Stand up the Romanian evaluation harness before the first model routing decision.
- Write the role-scoping spec before the first agent, not after the first security question.

What this case study demonstrates

- Designing AI systems for environments with hard compliance, audit, and data-residency constraints.
- Engineering discipline around role-scoping, tamper-evident logging, and structured outputs.
- An operator's instinct for which surfaces deserve AI confidence and which deserve a hard "refuse and route."

> *PrimarIA is the kind of public-sector AI most national e-government projects spend five years and €10M trying to ship. Building it as a single-engineer multi-tenant platform is not a stunt — it's the only way the unit economics work for a country of 3,000+ small municipalities.*